08. MCP 통합

이 장에서는 MCP(Model Context Protocol)를 통해 Hermes Agent의 기능을 확장하는 방법을 다룹니다. 빌트인 도구(4장)와 스킬(7장)만으로 대부분의 작업이 가능하지만, CLI가 없는 외부 서비스와 연결하거나 여러 에이전트가 같은 도구를 공유해야 할 때 MCP가 필요합니다. 언제 MCP를 쓰고, 언제 다른 접근법이 나은지도 함께 다룹니다.

이 장에서 다루는 내용

- MCP란 무엇인가
- MCP 서버 설정
- 실습: MCP로 기능 확장

08-1. MCP란 무엇인가

> ⚠️ ***부분 검증**: v0.16.0 소스(`tools/mcp_tool.py` , `tools/mcp_oauth.py` , `tools/osv_check.py` , `hermes_cli/mcp_config.py`) 분석으로 확인했습니다. 설정 스키마·OAuth·OSV 동작은 코드 기준이며, 실제 MCP 서버 연결로 실행 검증하지는 않았습니다.*

- Hermes Agent의 3계층 도구 구조
- 언제 MCP를 쓰고, 언제 안 쓸까
  - MCP가 적합한 경우
  - 빌트인 도구나 스킬이 더 나은 경우
- 서버 유형
  - Stdio (로컬 프로세스)
  - HTTP (원격 서버)
- 주요 기능
  - 도구 이름 규칙
  - 샘플링 (Sampling)
  - 설정 핫 리로드
  - OAuth 2.1 PKCE — MCP 서버 인증
  - OSV 패키지 멀웨어 스캔

MCP(Model Context Protocol)는 AI 에이전트를 외부 도구 서버와 연결하는 표준 프로토콜입니다. Anthropic이 2024년 말에 발표하고, 2025년 말에 Linux Foundation 산하 AAIF(Agentic AI Foundation)에 기증했습니다. OpenAI, Google 등도 채택하면서 "AI의 USB-C"로 불리고 있습니다.

GitHub, 데이터베이스, SaaS API 등 다양한 외부 서비스를 에이전트의 도구로 추가할 수 있으며, 한 번 만든 MCP 서버는 Claude, GPT, Hermes 등 어떤 에이전트에서든 사용할 수 있습니다.

Hermes Agent의 3계층 도구 구조

MCP를 이해하려면 먼저 Hermes Agent가 도구를 3계층으로 제공한다는 점을 알아야 합니다.



대부분의 일상적인 작업은 1~2계층(빌트인 도구 + 스킬)으로 충분합니다. MCP는 **외부 서비스와의 연결이 필요할 때** 사용하는 확장 메커니즘입니다.

언제 MCP를 쓰고, 언제 안 쓸까

2026년 초 현재, MCP의 사용처에 대한 커뮤니티 논의가 활발합니다. MCP가 만능은 아니며, 상황에 따라 더 나은 선택지가 있습니다.

MCP가 적합한 경우

- **CLI가 없는 SaaS 서비스**: Jira, Notion, Linear 등 웹 API만 제공하는 서비스
- **인증이 필요한 DB 연결**: PostgreSQL, MongoDB 등 상태 유지 연결
- **멀티 에이전트 환경**: 여러 에이전트가 같은 도구를 표준화된 방식으로 공유
- **엔터프라이즈 보안**: 다중 사용자 인증, 감사 추적이 필요한 조직

빌트인 도구나 스킬이 더 나은 경우

- **셸 명령어**: `git`, `docker`, `curl` 등은 LLM이 이미 사용법을 알고 있습니다. MCP로 감싸면 오히려 비효율적입니다.
- **반복 워크플로**: 배포 체크리스트, 코드 리뷰 절차 등은 스킬로 만드는 것이 간단합니다. 네트워크 호출도 필요 없습니다.

- **토큰 효율이 중요한 경우**: ScaleKit 벤치마크에 따르면 동일 작업에서 CLI는 약 1,400 토큰, MCP는 약 44,000 토큰을 소비합니다. MCP 서버의 도구 스키마를 로드하는 것만으로 수만 토큰이 소비될 수 있습니다.

판단 기준

"이 작업을 터미널 명령어 한 줄로 할 수 있는가?"라고 자문해 보세요. 가능하다면 빌트인 도구로 충분합니다. CLI가 없고, 인증이 복잡하고, 여러 에이전트가 같은 서비스를 써야 할 때 MCP를 도입하세요.

서버 유형

Stdio (로컬 프로세스)

로컬에서 서브프로세스로 실행됩니다. stdin/stdout으로 통신하므로 지연 시간이 짧습니다.

```
mcp_servers:
  github:
    command: "npx"
    args: ["-y", "@modelcontextprotocol/server-github"]
    env:
      GITHUB_PERSONAL_ACCESS_TOKEN: "${GITHUB_TOKEN}"
```

> 💡 ***전송 방식 자동 판별**: Hermes는 별도의 `type` 필드를 두지 않습니다. `command` 가 있으면 stdio, `url` 이 있으면 HTTP로 동작합니다(둘 다 있으면 `url` 을 우선).*

HTTP (원격 서버)

네트워크를 통해 원격 서버에 연결합니다.

```
mcp_servers:
  my-api:
    url: "https://api.example.com/mcp"
    headers:
      Authorization: "Bearer ${API_TOKEN}"
```

주요 기능

도구 이름 규칙

MCP 서버의 도구는 `mcp_<서버이름>_<도구이름>` 형태로 명명됩니다. 서버 이름이나 도구 이름에 포함된 하이픈(`-`)과 점(`.`)은 밑줄(`_`)로 자동 변환됩니다. 이렇게 해서 내장 도구와의 충돌을 방지합니다.

예: `mcp_github_create_issue` , `mcp_github_list_repos`

샘플링 (Sampling)

MCP는 서버가 클라이언트의 LLM에게 요청을 보내는 **샘플링** 기능을 지원합니다. 서버 측에서 LLM 추론이 필요한 경우, 사용자에게 별도 API 키를 요구하지 않고 클라이언트의 LLM을 활용할 수 있습니다. 이 기능은 MCP의 양방향 통신 구조를 보여주는 중요한 특징입니다.

설정 핫 리로드

MCP 설정을 변경한 후 에이전트를 재시작할 필요 없이 `/reload-mcp` 명령으로 즉시 반영할 수 있습니다. 새 서버를 추가하거나 도구 필터를 변경할 때 편리합니다. 다만 도구 정의가 바뀌면 프롬프트 캐시(시스템 프롬프트·툴 정의가 같다는 가정으로 비용을 줄이는 메커니즘)가 무효화됩니다 — `/reload-mcp` 는 그 영향을 안내하고 진행 여부를 한 번 더 확인합니다.

OAuth 2.1 PKCE — MCP 서버 인증

원격 MCP 서버가 OAuth로 보호되는 경우, Hermes는 표준 준수 OAuth 2.1 인증 코드 플로우(브라우저 + PKCE)를 지원합니다. 첫 사용 시 브라우저로 로그인 페이지가 열리고, 토큰 교환·동적 클라이언트 등록·갱신을 자동으로 처리합니다. 만료 토큰은 백그라운드에서 갱신됩니다.

OSV 패키지 멀웨어 스캔

stdio MCP 서버를 `npx` / `uvx` / `pipx` 같은 패키지 실행기로 띄울 때, 시작 전에 **OSV(Open Source Vulnerabilities) 데이터베이스에 패키지 이름·버전을 조회**합니다. 알려진 멀웨어가 발견되면 실행을 차단하고 경고합니다. `OSV_ENDPOINT` 환경 변수로 사설 미러를 지정할 수도 있습니다. 이 검사는 무료이며 추가 키가 필요하지 않습니다.

08-2. MCP 클라이언트 설정

- 전송 방식
- Stdio 서버 설정
- HTTP 서버 설정
- 필드별 의미
- 도구 필터링 (중요)
- 보안 고려사항
- 여러 서버 조합
- 동적 도구 갱신
- 입력 스키마 정합성 — definitions → $defs 자동 재작성 MCP 서버가 보내오는 도구 입력 스키마에는 JSON Schema 구버전 표기인 definitions 키가 종종 섞여 옵니다. 일부 LLM 프로바이더(주로 OpenAI 호환 엔드포인트)는 이를 거부하고 400 응답을 돌려줍니다. Hermes는 도구 등록 시 스키마의 definitions 참조를 표준 $defs로 자동 재작성해서 이런 호환성 문제를 회피합니다. 사용자가 손볼 일은 없습니다.

MCP 서버는 `~/.hermes/config.yaml` 의 `mcp_servers` 섹션에서 설정합니다.

전송 방식

MCP는 두 가지 전송 방식을 지원합니다.

Stdio (로컬 프로세스): `command` , `args` , `env` 필드를 사용합니다. 로컬에서 서브프로세스로 실행되며 stdin/stdout으로 통신합니다. 지연 시간이 짧아 대부분의 경우 권장됩니다.

HTTP (원격 서버): `url` , `headers` 필드를 사용합니다. 네트워크를 통해 원격 서버에 연결합니다. 클라우드 기반 MCP 서비스에 적합합니다.

Stdio 서버 설정

로컬 서브프로세스로 실행하는 MCP 서버는 다음 형식으로 설정합니다.

```
mcp_servers:
  서버이름:
    type: stdio
    command: "명령어"     # 실행할 바이너리 (예: "npx", "uvx")
    args: ["인수"]        # command에 전달할 인수 배열
    env:                  # 자식 프로세스에 전달할 환경변수
      KEY: "value"
    timeout: 60           # 도구 호출 타임아웃 (초, 기본 60)
    connect_timeout: 120  # 초기 연결 타임아웃 (초, 기본 120)
    enabled: true         # 활성화 여부
```

HTTP 서버 설정

원격 MCP 엔드포인트는 다음 형식으로 설정합니다.

```
mcp_servers:
  서버이름:
    type: http
    url: "https://mcp.example.com/sse"
    headers:
      Authorization: "Bearer ${MCP_TOKEN}"
    timeout: 60           # 도구 호출 타임아웃 (초, 기본 60)
    connect_timeout: 120  # 초기 연결 타임아웃 (초, 기본 120)
    enabled: true         # 활성화 여부
```

필드별 의미

필드	적용	설명
`type`	모두	`stdio` 또는 `http`
`command`	stdio	실행할 바이너리 (`npx` , `uvx` , 직접 빌드한 실행 파일 등)
`args`	stdio	command에 넘길 인수 배열
`env`	stdio	자식 프로세스에 전달할 환경변수. 호스트 환경변수는 자동으로 노출되지 않음
`url`	http	MCP 엔드포인트 URL
`headers`	http	HTTP 요청 헤더(인증 토큰 등)
`timeout`	모두	개별 도구 호출 타임아웃, 초 단위 (기본 60)
`connect_timeout`	모두	서버 초기 연결 타임아웃, 초 단위 (기본 120)
`enabled`	모두	`false` 이면 해당 서버를 로드하지 않음
`tools.include` / `tools.exclude`	모두	도구 허용 목록·차단 목록 (아래 「도구 필터링」 참고)
`resources`	모두	`false` 이면 `list_resources` · `read_resource` 비활성화

도구 필터링 (중요)

MCP 서버는 수십 개의 도구를 한꺼번에 노출하는 경우가 많습니다. 예를 들어 GitHub MCP 서버는 30개 이상의 도구를 제공합니다. 이 도구들의 스키마가 모두 컨텍스트 윈도우에 로드되므로, 필요한 도구만 필터링하는 것이 **토큰 절약과 성능 향상**에 중요합니다.

```
mcp_servers:
  github:
    type: stdio
    command: "npx"
    args: ["-y", "@modelcontextprotocol/server-github"]
    tools:
      include:            # 이 도구만 사용
        - create_issue
        - list_repos
      # 또는
      exclude:            # 이 도구 제외
        - delete_repo
    resources: false      # list_resources, read_resource 비활성화
```

`include`가 설정되어 있으면 `exclude`보다 우선합니다. 두 필드를 동시에 설정하면 `include`만 적용됩니다.

`resources: false`를 설정하면 `list_resources`와 `read_resource` 도구가 비활성화됩니다. 리소스 접근이 불필요한 서버에서 토큰을 추가로 절약할 수 있습니다.

토큰 절약 팁

로컬 LLM을 사용할 때는 컨텍스트 윈도우가 제한적이므로 도구 필터링이 특히 중요합니다. 실제로 필요한 도구 2~3개만 허용 목록에 등록하세요. MCP 서버를 여러 개 연결할수록 토큰 소비가 급격히 늘어납니다.

보안 고려사항

Hermes Agent는 MCP 서버의 보안을 여러 계층으로 관리합니다.

환경변수 필터링: Stdio 서버에는 안전한 기본값(PATH, HOME, USER, LANG 등)과 사용자가 `env` 필드에 명시한 변수만 전달됩니다. 호스트의 전체 환경변수가 노출되지 않으므로 민감 정보 유출 위험이 줄어듭니다.

크리덴셜 스트리핑: 에러 메시지에서 PAT, API 키, Bearer 토큰 등 인증 정보가 자동으로 제거됩니다. 에러 로그가 외부에 노출되더라도 토큰이 유출되지 않습니다.

자동 재연결: MCP 서버 연결이 끊어지면 지수 백오프(exponential backoff) 방식으로 자동 재연결을 시도합니다. 최대 5회까지 재시도하며, 모두 실패하면 해당 서버를 비활성화합니다.

- 신뢰할 수 없는 MCP 서버는 사용하지 마세요
- 서버별 타임아웃을 설정하여 응답 없는 서버가 에이전트를 블로킹하지 않도록 하세요

여러 서버 조합

여러 MCP 서버를 동시에 연결할 수 있습니다.

```
mcp_servers:
  github:
    type: stdio
    command: "npx"
    args: ["-y", "@modelcontextprotocol/server-github"]
    env:
      GITHUB_TOKEN: "${GITHUB_TOKEN}"

  postgres:
    type: stdio
    command: "npx"
    args: ["-y", "@modelcontextprotocol/server-postgres"]
    env:
      DATABASE_URL: "${DATABASE_URL}"

  filesystem:
    type: stdio
    command: "npx"
    args: ["-y", "@modelcontextprotocol/server-filesystem", "/path/to/allow"]
```

동적 도구 갱신

연결된 MCP 서버가 도구 목록 변경을 알리면(ToolListChangedNotification), Hermes는 자동으로 도구 목록을 다시 가져옵니다. 새로 추가된 도구는 등록되고 제거된 도구는 해제되므로, MCP 서버 쪽에서 도구가 변경되더라도 Hermes를 재시작할 필요가 없습니다.

입력 스키마 정합성 — `definitions` → `$defs` 자동 재작성 MCP 서버가 보내오는 도구 입력 스키마에는 JSON Schema 구버전 표기인 `definitions` 키가 종종 섞여 옵니다. 일부 LLM 프로바이더(주로 OpenAI 호환 엔드포인트)는 이를 거부하고 400 응답을 돌려줍니다. Hermes는 도구 등록 시 스키마의 `definitions` 참조를 표준 `$defs`로 자동 재작성해서 이런 호환성 문제를 회피합니다. 사용자가 손볼 일은 없습니다.

08-3. 실습: MCP로 기능 확장

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스를 기준으로 점검했습니다. MCP 설정 형식(`mcp_servers` 블록, 전송 방식 자동 판별)·도구 접두사(`mcp_`)·`/reload-mcp` · `/tools` 명령은 소스와 대조해 확인했고, 실습 절차의 실제 실행 결과는 직접 실행 테스트하지 않았습니다.*

- 실습 1: 파일시스템 MCP 서버
- 실습 2: 도구 확인
- 실습 3: 같은 작업, 다른 접근법 비교
  - 방법 1: 빌트인 도구 (터미널)
  - 방법 2: MCP 파일시스템 서버
  - 방법 3: 스킬로 자동화
  - 비교 결과
- MCP 서버 직접 만들기

이 절에서는 실제로 MCP 서버를 설정하여 Hermes Agent의 기능을 확장합니다.

실습 1: 파일시스템 MCP 서버

가장 간단한 MCP 서버인 파일시스템 서버를 연결해 봅니다.

`~/.hermes/config.yaml`에 추가:

```
mcp_servers:
  filesystem:
    command: "npx"
    args: ["-y", "@modelcontextprotocol/server-filesystem", "/tmp/mcp-test"]
    enabled: true
```

> 💡 ***전송 방식 자동 판별**: `command`가 있으면 stdio, `url`이 있으면 HTTP로 자동 판별됩니다. 별도의 `type` 필드는 사용하지 않습니다.*

설정을 저장한 후 `/reload-mcp` 명령을 실행하면 재시작 없이 MCP 서버가 로드됩니다(실행 시 프롬프트 캐시 무효화 경고와 함께 확인을 한 번 거칩니다). 또는 Hermes Agent를 재시작해도 됩니다.

```
> /tools    # mcp_filesystem_* 도구가 보이는지 확인
> mcp 파일시스템 도구로 /tmp/mcp-test에 테스트 파일을 만들어줘
```

실습 2: 도구 확인

연결된 MCP 서버의 도구 목록을 확인합니다.

```
> 현재 사용 가능한 MCP 도구 목록을 알려줘
```

또는 직접:

```
> /tools
```

MCP 도구는 `mcp_` 접두사로 표시됩니다.

실습 3: 같은 작업, 다른 접근법 비교

Hermes Agent의 3계층 도구 구조를 체감하기 위해, 동일한 작업을 세 가지 방법으로 수행해 봅니다.

과제: `/tmp/mcp-test` 디렉터리의 파일 목록을 확인하고, 새 파일을 만들기

방법 1: 빌트인 도구 (터미널)

```
> /tmp/mcp-test 디렉터리의 파일 목록을 보여주고, hello.txt를 만들어줘
```

에이전트가 `ls` 와 `echo > hello.txt` 같은 셸 명령을 직접 실행합니다. 별도 설정이 필요 없고, LLM이 이미 이 명령어들을 알고 있으므로 가장 빠릅니다.

방법 2: MCP 파일시스템 서버

실습 1에서 설정한 MCP 서버를 사용합니다.

```
> mcp 파일시스템 도구로 /tmp/mcp-test의 파일 목록을 보여주고, hello-mcp.txt를 만들어줘
```

같은 결과를 얻지만, MCP 프로토콜을 통한 도구 호출이 추가됩니다.

방법 3: 스킬로 자동화

이 작업을 자주 한다면 스킬로 만들 수 있습니다.

```
> 방금 한 파일 관리 작업을 스킬로 만들어줘
```

에이전트가 절차를 `SKILL.md`로 저장하고, 다음부터는 `/file-manager` 같은 명령으로 빠르게 실행할 수 있습니다.

비교 결과

	빌트인 도구	MCP	스킬
설정	불필요	config.yaml 수정	한 번 생성
토큰 소비	최소	도구 스키마 로드 필요	필요 시에만 로드
적합한 상황	대부분의 로컬 작업	CLI 없는 외부 서비스	반복되는 워크플로

이 실습을 통해 "어떤 계층을 선택할지"에 대한 감각을 익힐 수 있습니다.

MCP 서버 직접 만들기

Python으로 간단한 MCP 서버를 만들 수도 있습니다. 이 부분은 고급 주제이므로 공식 문서를 참조하세요.

설정 변경 시 /reload-mcp

MCP 설정을 수정할 때마다 에이전트를 재시작할 필요가 없습니다. `/reload-mcp` 명령으로 변경 사항을 즉시 반영할 수 있습니다. 도구 필터를 조정하거나 새 서버를 추가할 때 특히 편리합니다.

MCP 서버 생태계

MCP 프로토콜은 빠르게 성장하고 있어, 다양한 서비스에 대한 MCP 서버가 이미 존재합니다. GitHub, Slack, PostgreSQL, MongoDB, Notion, Linear 등에 대한 서버를 npm에서 찾을 수 있습니다. 단, 서버를 추가할수록 토큰 소비가 늘어나므로, 실제로 필요한 서버만 연결하는 것이 좋습니다.

> ***주의: 로컬 LLM과 MCP 사용 시 알려진 문제*** *로컬 LLM(예: Ollama 기반 모델)에서 MCP를 사용할 때, 도구 호출이 텍스트로 출력되는 버그가 보고되어 있습니다(GitHub #741). 도구 호출 JSON이 응답 텍스트에 그대로 노출되는 현상입니다. 로컬 LLM으로 MCP를 사용할 때는 이 점을 참고하세요.*

08-4. Hermes를 MCP 서버로 노출하기

- 시작
- 제공 도구 (10개)
- 타임아웃 대처 아키텍처
  - 즉시 반환 도구
  - 장기 폴링 (events_wait)
  - EventBridge
- 승인 메커니즘

Hermes Agent는 MCP 클라이언트(08-2)로 외부 도구를 사용하는 것 외에, 자체가 **MCP 서버**로 작동할 수도 있습니다. 이렇게 하면 Claude Code, Cursor, Codex 등 MCP 클라이언트를 지원하는 도구에서 Hermes의 대화 세션에 접근할 수 있습니다.

시작

```
hermes mcp serve
```

stdio 기반 MCP 서버로 시작됩니다. MCP 클라이언트 설정에서 위 명령을 stdio 서버로 등록하면 됩니다.

```
{
    "mcpServers": {
        "hermes": {
            "command": "hermes",
            "args": ["mcp", "serve"]
        }
    }
}
```

제공 도구 (10개)

도구	기능
`conversations_list`	플랫폼별 대화 목록 조회
`conversation_get`	특정 대화의 상세 정보
`messages_read`	대화의 메시지 이력 읽기
`messages_send`	대화에 메시지 전송
`attachments_fetch`	메시지의 첨부 파일 조회
`events_poll`	새 이벤트를 즉시 반환 (비차단)
`events_wait`	새 이벤트를 대기 (장기 폴링)
`channels_list`	메시지를 보낼 수 있는 채널 목록
`permissions_list_open`	미결 승인 요청 목록
`permissions_respond`	승인 요청에 응답

타임아웃 대처 아키텍처

MCP 클라이언트는 보통 도구 호출에 타임아웃을 설정합니다(예: 60초). Hermes MCP 서버는 이 제약 안에서 작동하도록 설계되어 있습니다.

즉시 반환 도구

`conversations_list`, `messages_read`, `events_poll` 등 대부분의 도구는 로컬 DB/파일을 직접 읽어 **즉시 반환**합니다. 타임아웃 걱정이 없습니다.

장기 폴링 (events_wait)

`events_wait`는 새 이벤트가 올 때까지 대기하는 도구입니다. `timeout_ms` 파라미터로 대기 시간을 지정하되, 내부적으로 **5분(300,000ms) 상한**이 적용됩니다. MCP 클라이언트 타임아웃보다 짧게 설정하면 타임아웃 전에 응답을 받을 수 있습니다.

```
events_wait(timeout_ms=30000)  → 최대 30초 대기 후 응답
```

기본값은 30초(30,000ms)로, 대부분의 MCP 클라이언트 타임아웃(60초) 안에 수용됩니다.

EventBridge

백그라운드 스레드가 200ms 간격으로 SessionDB를 폴링하여 새 메시지와 승인 요청을 이벤트 큐(최대 1,000건)에 축적합니다. 파일 mtime 캐시를 사용해 변경이 없으면 비용 없이 건너뛰므로, 200ms 폴링이 실질적으로 부담이 되지 않습니다.

```
MCP 클라이언트 ←(stdio)→ FastMCP 서버 ←(이벤트큐)→ EventBridge ←(200ms 폴링)→ SessionDB
```

승인 메커니즘

Hermes를 MCP 서버로 노출하면 외부 AI 도구가 메신저 대화에 접근하고 메시지를 보낼 수 있게 됩니다. 과도한 권한 남용을 방지하기 위해 민감한 작업에는 승인이 필요합니다.

승인 흐름:

1. 외부 클라이언트가 메시지 전송 등 민감한 작업을 시도합니다
2. Hermes가 `approval_requested` 이벤트를 발생시킵니다
3. 클라이언트가 `permissions_list_open`으로 대기 중인 승인 요청을 확인합니다
4. `permissions_respond`로 `allow-once`, `allow-always`, 또는 `deny`를 응답합니다

대화 목록 조회나 메시지 읽기 같은 읽기 작업은 승인 없이 가능하지만, 메시지 전송 같은 쓰기 작업은 승인을 거치도록 설계되어 있습니다.